

ActivityResult (Galeria) e Filtro para RecyclerView

Professor: Henrique Galati
IFSP - Campus Araraquara
Análise e Desenvolvimento de Sistemas

Até agora, nosso aplicativo `AgendaApp` exibia as imagens dos contatos utilizando recursos internos da pasta `res/drawable`, como `R.drawable.minkey`. Esse método funciona bem para imagens pré-definidas na instalação do app. No entanto, ele apresenta uma limitação importante: **não é possível acessar ou armazenar imagens da galeria do usuário** com `R.drawable`. Esses recursos são compilados em tempo de build e não podem ser modificados ou expandidos dinamicamente.

Com isso, surge a necessidade de aceitar imagens externas — como uma foto tirada com a câmera ou escolhida da galeria. Para isso, precisamos representar as imagens de outra forma no app: **através da classe `Uri`**. Já utilizamos a classe `Uri` para o uso das Intents implícitas (chamada para o telefone, site, Google Maps...)

A classe `Uri` (Uniform Resource Identifier) representa um caminho para um recurso — no nosso caso, uma imagem no dispositivo Android. Pode apontar para arquivos da galeria, imagens baixadas, fotos tiradas pela câmera etc.

Exemplos de `Uri`:

- `content://media/external/images/media/12345`
- `file:///storage/emulated/0/Pictures/meufoto.jpg`

Refatorando a Classe `Contato`

A classe `Contato` antes armazenava a foto com um `Int`, representando o ID do recurso (ex: `R.drawable.minkey`). Para suportar imagens externas, vamos substituir esse campo por uma `String`, que pode referenciar imagens tanto da galeria quanto da câmera:

```
data class Contato(
    val foto: String,
    val nome: String,
    val telefone: String,
    val email: String
) : Serializable
```

Por que usar uma String ao invés da classe Uri na foto?

A classe `Uri`, embora ideal para representar endereços de recursos como imagens e arquivos no Android, **não é serializável por padrão**, o que significa que não pode ser diretamente enviada entre `Activities` usando `Intent.putExtra` com `Serializable`. Isso gera erros de execução ao tentar transportar objetos contendo `Uri`, como a nossa classe `Contato`. Para contornar esse problema de forma prática e segura, optamos por armazenar o `Uri` como uma **string** (usando `uri.toString()` na origem e `uriString.toUri()` ao recuperar).

Atualizando a tela de cadastro para usar imagem da galeria

Após refatorar a classe `Contato`, podemos preparar a `CadastroContatoActivity` para permitir que o usuário selecione uma imagem da galeria ao clicar na imagem padrão do formulário. Para isso, utilizaremos um `ActivityResultLauncher` com o contrato `ActivityResultContracts.OpenDocument()`.

```
class CadastroContatoActivity : AppCompatActivity() {

    private lateinit var binding: ActivityCadastroBinding
    private lateinit var launcherGaleria: ActivityResultLauncher<Array<String>>
    private lateinit var uriSelecionada: String

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityCadastroBinding.inflate(layoutInflater)
        setContentView(binding.root)

        setupLauncher()
        setupListeners()
    }
}
```

```

private fun setupLauncher(){
    launcherGaleria = registerForActivityResult(
        ActivityResultContracts.OpenDocument()
    ) { uri ->
        if (uri != null) {
            contentResolver.takePersistableUriPermission(
                uri,
                Intent.FLAG_GRANT_READ_URI_PERMISSION
            )

            uriSelecionada = uri.toString()
            binding.imgFoto.setImageURI(uri)
        }
    }
}

private fun setupListeners() {
    binding.imgFoto.setOnClickListener {
        launcherGaleria.launch(arrayOf("image/*"))
    }

    binding.btnSalvar.setOnClickListener {
        val foto = uriSelecionada
        val nome = binding.edtNome.text.toString()
        val telefone = binding.edtTelefone.text.toString()
        val email = binding.edtEmail.text.toString()

        if (nome.isNotBlank() && telefone.isNotBlank() && email.isNotBlank()) {
            val novoContato = Contato(
                foto = foto,
                nome = nome,
                telefone = telefone,
                email = email
            )

            val resultIntent = Intent().apply {
                putExtra("contato", novoContato)
            }

            setResult(Activity.RESULT_OK, resultIntent)
            finish()
        }
        else{
            Toast.makeText(this, "Preencha todos os campos",
                Toast.LENGTH_SHORT).show()
        }
    }

    binding.btnVoltar.setOnClickListener {
        setResult(Activity.RESULT_CANCELED)
        finish()
    }
}
}

```

A função `setupLauncher()` é responsável por configurar o processo de abertura da galeria de imagens do dispositivo e capturar a imagem selecionada pelo usuário. Para isso, ela utiliza a **API de Activity Result**, mais moderna e segura que o antigo `startActivityForResult`. Especificamente, ela usa o contrato `ActivityResultContracts.OpenDocument()`, que permite abrir documentos do sistema de arquivos, incluindo imagens da galeria.

Ao registrar esse *launcher*, definimos um callback (lambda) que será chamado quando o usuário selecionar uma imagem. Se a `uri` retornada não for nula, o aplicativo solicita uma **permissão persistente** de leitura (`takePersistableUriPermission`) sobre aquele URI. Isso é essencial para garantir que o app continue tendo acesso à imagem mesmo após o reinício do dispositivo ou da aplicação. A URI da imagem é então convertida para string (e armazenada na variável `uriSelecionada`) e usada para atualizar a visualização da `ImageView` (`imgFoto`) com `setImageURI()`.

Essa abordagem é importante porque o Android impõe restrições cada vez maiores ao acesso direto a arquivos no armazenamento. Usar URIs com permissão persistente garante que a imagem permaneça acessível de forma segura e compatível com futuras versões do sistema.

Removendo o método `loadData()` da `MainActivity`

Agora que os contatos serão adicionados apenas pelo usuário (e suas fotos virão da galeria), o método `loadData()` com dados simulados e imagens fixas de `R.drawable` deixa de fazer sentido e pode ser removido. Isso deixa a `MainActivity` mais limpa e realista, pronta para manipular dados reais inseridos em tempo de execução. Apague o método `loadData()` e em seu lugar, adicione a inicialização da lista vazia:

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    binding = ActivityMainBinding.inflate(layoutInflater)
    setContentView(binding.root)

    contatos = mutableListOf()
    setupRecyclerView()
    setupLauncherCadastro()
    setupListeners()
}
```

Refatorando o Adapter

Por fim, para que o `RecyclerView` funcione corretamente, precisaremos refatorar o método `bind` da classe interna de nosso `Adapter`:

```
inner class ViewHolder(val binding: ItemContatoBinding) :
    RecyclerView.ViewHolder(binding.root) {
        fun bind(contato: Contato) {
            binding.imgFoto.setImageURI(contato.foto.toUri())
            binding.tvNome.text = contato.nome
            binding.tvTelefone.text = contato.telefone
            binding.tvEmail.text = contato.email
            binding.root.setOnClickListener { onClick(contato) }
        }
    }
```

Refatorando a classe `DetalheContatoActivity`

Por fim, é necessário alterar esta classe para receber a `String` do objeto `contato`, transformá-la em `Uri` e usá-la na imagem, assim como fizemos no `ViewHolder`:

```
private fun setupViews()
{
    binding.tvNome.text = contato.nome
    binding.tvTelefone.text = contato.telefone
    binding.tvEmail.text = contato.email
    binding.imgFoto.setImageURI(contato.foto.toUri())
}
```

Agora entre no navegador ou câmera do emulador e salve uma foto na galeria. Após isso, teste seu aplicativo.

Adicionando busca por nomes no `RecyclerView`

Nesta última parte, vamos adicionar um `EditText` entre o botão de adicionar contato e o `RecyclerView`:

```
<EditText
    android:id="@+id/edtFiltro"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
```

```

        android:hint="Filtrar por nome"
        android:inputType="text"
        android:drawableStart="@android:drawable/ic_menu_search"
        android:drawablePadding="8dp"
        android:padding="12dp"
        android:background="@android:drawable/edit_text"
        android:textSize="16sp"
        android:layout_marginTop="12dp"
        app:layout_constraintTop_toBottomOf="@id/btnAdicionar"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent" />

```

Agora, precisaremos adicionar um listener para este `EditText` que, ao modificar o texto dentro dele, será aplicado um filtro na nossa lista de contatos, enviadas para o `Adapter` e atualizadas no `RecyclerView`:

```

binding.edtFiltro.addTextChangedListener(object : TextWatcher {
    override fun beforeTextChanged(s: CharSequence?, start: Int, count: Int, after: Int) {}

    override fun onTextChanged(s: CharSequence?, start: Int, before: Int, count: Int) {
        val filtro = s.toString().lowercase()
        val filtrados = contatos.filter {
            it.nome.lowercase().contains(filtro)
        }

        adapter.updateLista(filtrados)
    }

    override fun afterTextChanged(s: Editable?) {}
})

```

Aqui, utilizamos o método `addTextChangedListener` com uma implementação anônima da interface `TextWatcher`. O `TextWatcher` é uma interface do Android usada para observar mudanças no texto de campos editáveis como `EditText`. Ele é parte da API de eventos de interface gráfica do Android e permite que o desenvolvedor **monitore em tempo real** três fases de uma alteração de texto:

- `beforeTextChanged(...)`: chamado **antes** do texto mudar.
- `onTextChanged(...)`: chamado **enquanto** o texto está mudando.
- `afterTextChanged(...)`: chamado **depois** que o texto foi alterado.

No Kotlin, quando usamos `object : Interface { ... }`, estamos criando uma **classe anônima** que **implementa uma interface**. Essa é uma forma concisa de dizer: “Crie

uma classe que implementa `TextWatcher`, mas não precisa ter nome, e eu vou definir os métodos aqui mesmo.” Essa sintaxe é **útil e comum** para eventos de UI, pois evita que você crie uma nova classe só para algo simples como ouvir alterações em um campo de texto.

A linha `val filtrados = contatos.filter { it.nome.lowercase().contains(filtro) }` aplica um filtro sobre a lista original de contatos, retornando uma nova lista contendo apenas os elementos cujo nome corresponde ao texto buscado. O método `filter` é uma função da classe `List` em Kotlin e faz parte das funções de extensão da API de coleções. Ele recebe uma *lambda* como parâmetro — ou seja, uma função anônima — que define a condição de filtragem: no caso, `it.nome.lowercase().contains(filtro)`. Essa condição verifica se o nome do contato (convertido para minúsculas com `lowercase()`) contém a string digitada pelo usuário. Cada elemento da lista é representado por `it` dentro da lambda, e apenas aqueles que satisfazem a condição retornam `true` e são incluídos na nova lista.

Por fim, iremos refatorar nossa classe `Adapter`, com o método `updateLista`:

```
fun updateLista(novosContatos: List<Contato>) {  
    this.contatos = novosContatos  
    notifyDataSetChanged()  
}
```

Teste seu aplicativo com estas novas funcionalidades.

Exercício de Fixação – Cadastro de Serviços

Implemente a refatoração aplicada ao AgendaApp em seu aplicativo de listagem de serviços usados como projeto avaliativo.